

Foundry e metodo di testing

Dai requisiti alle proprietà verificabili

`setUp` · `vm.prank` · `vm.deal` · `vm.expectRevert`

Ambiente esclusivamente didattico, difensivo e locale:
contratti giocattolo, account di test, fondi fittizi.

Baseline: Solidity 0.8.37
verificata il 21 settembre 2026

1 • OBIETTIVI

Cosa saprai fare alla fine della lezione

Organizzare il laboratorio

Progetto Foundry minimale, contratto sotto test e test harness, forge build e forge test con filtri, trace e debugger.

Costruire il contesto

forge-std/Test.sol, setUp() per uno stato iniziale ripetibile, vm.prank per il chiamante, vm.deal per i fondi fittizi.

Verificare i fallimenti

vm.expectRevert, test positivi e negativi, e nessun effetto parziale lasciato da un revert.

Giudicare la suite

Quali bug scoprirebbe davvero? Dalla specifica ai test, prima di chiamare «corretto» un contratto.

LA DISCIPLINA DI TUTTO IL CORSO

Ogni lezione seguirà la stessa sequenza, dalla specifica al test che impedisce di tornare indietro.

1 Specifica	2 Proprietà	3 Implementazione	4 Test positivi
5 Test negativi	6 Tentativo di violazione	7 Correzione	8 Test di regressione

2 · MODELLO MENTALE

Dal requisito al test, in tre livelli

1 · REQUISITO

«Un indirizzo può registrarsi una sola volta pagando esattamente 1 ETH»



2 · PROPRIETÀ VERIFICABILI

P1 · 1 ETH esatto, nuovo utente: successo
P2 · meno di 1 ETH: revert
P3 · più di 1 ETH: revert
P4 · secondo tentativo, stesso utente: revert
P5 · dopo un revert la contabilità non cambia



3 · TEST FOUNDRY

Una funzione Solidity per ogni comportamento importante.

`test_RevertWhen_FeeIsTooHigh`

I test derivano dalla specifica, non viceversa.

2 · IL RAGIONAMENTO OPPOSTO

Il rischio: testare il codice contro se stesso

Se scrivi prima il contratto e poi inventi test che confermano ciò che fa già, stai testando l'implementazione contro se stessa.

UN AUDITOR RAGIONA AL CONTRARIO

- 1** Cosa DEVE essere vero?
- 2** Quali stati o input potrebbero renderlo falso?
- 3** Quale test distingue l'implementazione corretta da quella sbagliata?

2 • IL TEST COME ESPERIMENTO

In una EVM locale, il test controlla tutto



Distribuisce contratti



Assegna bilanci



Sceglie msg.sender per la prossima chiamata



Chiama con msg.value



Legge lo storage tramite getter



Si aspetta un revert preciso



Ispeziona le tracce



Confronta atteso e osservato

Non stiamo fingendo il comportamento di Solidity: il bytecode viene compilato ed eseguito secondo la semantica EVM, in una EVM locale gestita da Foundry.

2 • TEST INDIPENDENTI

setUp() riparte da capo prima di ogni test



3.1 • LA TOOLCHAIN

Foundry: quattro strumenti, oggi quasi solo Forge

Forge

build, test, fuzzing,
invariant testing,
script, debug

La useremo oggi

Anvil

nodo Ethereum locale

Cast

interazione da CLI con
ABI, RPC, chiamate e
transazioni

Chisel

REPL Solidity

Il laboratorio resta interamente locale: nessun RPC esterno.

3.2 • STRUTTURA E NAMING

Il nome del test dice quale requisito protegge

```
lesson-02/  
├─ foundry.toml  
├─ src/  
│   └─ FixedFeeRegistry.sol  
├─ test/  
│   └─ FixedFeeRegistry.t.sol  
└─ lib/  
    └─ forge-std/
```

I file di test finiscono spesso in **.t.sol**. Forge considera test le funzioni il cui nome inizia con **test**.

```
function test_Register() public { ... }  
function test_RevertWhen_FeeIsWrong() public { ... }
```

Naming suggerito dalla documentazione Foundry:

`test_RevertIf_Condition`

`test_RevertWhen_Condition`

3.3 • FORGE-STD/TEST.SOL

Test: assertion, vm e utility

```
import {Test} from "forge-std/Test.sol";

contract ExampleTest is Test {
    // ...
}
```

```
vm.deal(alice, 10 ether);
```

COSA METTE A DISPOSIZIONE

Assertion come `assertEq`, `assertTrue`, `assertFalse`

L'istanza **vm**, per invocare i cheatcode

Utility della standard library di Foundry

Un cheatcode configura la EVM di test: qui imposta a 10 ether il saldo di alice nello stato locale. Non chiama il contratto e on-chain non esiste.

Mai confondere le capacità del test harness con le capacità reali di un attaccante.

3.4 • ARRANGE → ACT → ASSERT

Un test leggibile ha tre fasi

```
function test_Register_SucceedsWithExactFee() public {  
  // Arrange  
  vm.deal(ALICE, 10 ether);  
  
  // Act  
  vm.prank(ALICE);  
  registry.register{value: FEE}();  
  
  // Assert  
  assertTrue(registry.registered(ALICE));  
}
```

UN TEST DI SICUREZZA RENDE EVIDENTE

1. quale stato iniziale costruisce
2. quale identità controlla la chiamata
3. quale input viene inviato
4. cosa dovrebbe succedere
5. cosa non deve cambiare

Non è una regola sintattica: è una disciplina di leggibilità.

vm.prank sceglie chi chiama, vm.deal con quali fondi

vm.prank

Senza prank, il chiamante visto dal contratto è di norma il contratto di test stesso.

```
vm.prank(ALICE);  
registry.register{value: 1 ether}();
```

Cambia msg.sender per la **prossima chiamata rilevante**.
Per più chiamate: startPrank e stopPrank, più avanti.

vm.deal

Per inviare Ether, l'account di test deve avere saldo sufficiente.

```
vm.deal(ALICE, 10 ether);  
// deal(address account, uint256 newBalance)
```

Imposta il saldo al valore indicato. Non è un faucet: non crea ETH su Ethereum e non tocca alcun wallet.

Il comportamento deve essere corretto per identità diverse, non solo per il contratto di test.

3.7 · VM.EXPECTREVERT

Il revert atteso va verificato con precisione

Qualsiasi revert

debole: passa per ogni motivo

```
vm.expectRevert();
```

Errore senza argomenti

il selector dell'errore

```
vm.expectRevert(CustomError.selector);
```

Errore con argomenti

più forte: revert data completo

```
vm.expectRevert(abi.encodeWithSelector(  
    FixedFeeRegistry.ExactFeeRequired.selector, sent, FEE));
```

Un test troppo generico può passare per la ragione sbagliata: se il contratto reverte prima, per un bug diverso, expectRevert() dichiara comunque successo.

3.8 · TRE TIPI DI TEST

Positivo, negativo, di regressione

Positivo

Verifica che un comportamento valido sia possibile.

Alice paga 1 ETH → registrata

Negativo

Verifica che un comportamento vietato non sia possibile.

Alice paga 0.5 ETH → revert
Alice paga 2 ETH → revert
secondo tentativo → revert

Di regressione

Nasce da un bug specifico: prima lo riproduce, dopo la correzione resta nella suite.

IL CICLO DEL TEST DI REGRESSIONE

1 · bug osservato

2 · scrivo un test che fallisce

3 · correggo il contratto

4 · lo stesso test passa

5 · il test resta nella suite

3.9 • EFFETTI COLLATERALI

Dopo un revert, controlla anche cosa non è cambiato

Il contatore è rimasto invariato?

Il mapping è rimasto invariato?

La contabilità è rimasta invariata?

L'ETH è stata restituita dal revert?

UN BUON TEST NEGATIVO VERIFICA ENTRAMBE LE COSE

1 • l'operazione **non** ha avuto successo

2 • lo stato dopo il fallimento è quello atteso

Deriva dall'atomicità EVM vista nella Lezione 1.

3.10 · COVERAGE

Coverage utile, ma non equivalente a sicurezza

100% coverage $\neq$ 100% correttezza

100% coverage $\neq$ 100% sicurezza

```
if (msg.value < 1 ether) revert();
```

Entrambi i branch possono essere coperti. Ma se il requisito è **esattamente** 1 ETH, il caso 2 ether potrebbe non essere mai stato testato: codice coperto, requisito non verificato.

Usala come domanda, «quali rami non ho mai esercitato?», non come prova di sicurezza.

«Un indirizzo può registrarsi una sola volta pagando esattamente 1 ether.»

Niente prelievi né chiamate esterne: arriveranno più avanti.

IL CONTRATTO CONSERVA

se un indirizzo è registrato

il numero di registrazioni riuscite

il totale delle fee contabilizzate

FixedFeeRegistry: costante, stato, errori, evento

```
// pragma solidity 0.8.37;  
uint256 public constant REGISTRATION_FEE = 1 ether;  
  
mapping(address account ⇒ bool isRegistered) public registered;  
  
uint256 public registrationCount;  
uint256 public totalReceived;  
  
error ExactFeeRequired(uint256 sent, uint256 required);  
error AlreadyRegistered(address account);  
  
event Registered(address indexed account, uint256 amount);
```

PICCOLO, MA CONTIENE GIÀ

msg.sender

msg.value

mapping in storage

variabili persistenti

custom errors

revert

evento

una proprietà contabile

casi positivi e negativi non banali

4 • LA FUNZIONE CHE SCRIVE

register: prima i due check, poi lo storage

```
function register() external payable {  
    if (registered[msg.sender]) {  
        revert AlreadyRegistered(msg.sender);  
    }  
  
    if (msg.value != REGISTRATION_FEE) {  
        revert ExactFeeRequired(msg.value, REGISTRATION_FEE);  
    }  
  
    registered[msg.sender] = true;  
    registrationCount += 1;  
    totalReceived += msg.value;  
  
    emit Registered(msg.sender, msg.value);  
}
```

1 • DUE CHECK

Già registrato? Poi: fee esatta? Entrambi prima di ogni scrittura.

2 • TRE SCRITTURE

registered, registrationCount, totalReceived.

3 • UN EVENTO

Un log: nessun controllo ceduto a codice esterno.

5.1 • ANALISI DEL CODICE

register letta come la legge un auditor

Domanda	Risposta
Chi può chiamarla?	Chiunque: nessun access control, perché il requisito lo consente
Input del chiamante	Nessun parametro esplicito, ma il chiamante controlla msg.sender e msg.value
Valore in ingresso	payable; accettato solo msg.value == 1 ether: non almeno, non fino a, esattamente
Stato letto	registered[msg.sender] e la costante REGISTRATION_FEE
Stato scritto	Solo in caso di successo: registered[msg.sender], registrationCount, totalReceived
Chiamate esterne	Nessuna: l'emit produce un log, il controllo non passa mai a codice esterno

«Esattamente» non è un dettaglio: sarà il centro del mutation lab.

Cinque assunzioni che la suite deve verificare

- A1 una registrazione valida richiede la fee esatta
- A2 un account può registrarsi una sola volta
- A3 ogni registrazione riuscita incrementa il conteggio di uno
- A4 ogni registrazione riuscita incrementa totalReceived di 1 ether
- A5 un revert non deve lasciare modifiche persistenti

GETTER GENERATI DA PUBLIC

```
registered(ALICE)  
registrationCount()  
totalReceived()  
REGISTRATION_FEE()
```

Osservano lo stato senza aggiungere al contratto funzioni scritte solo per i test.

6 · PROPRIETÀ E INVARIANTI

Il requisito, tradotto in sei proprietà

ID	Proprietà	Enunciato
P1	Fee esatta	Una registrazione riesce soltanto se <code>msg.value == REGISTRATION_FEE</code>
P2	Registrazione unica	Dopo un successo, lo stesso indirizzo non può registrarsi di nuovo
P3	Identità indipendenti	La registrazione di Alice non impedisce a Bob di registrarsi
P4	Contatore coerente	<code>registrationCount</code> cresce di uno per ogni successo, non cambia se la chiamata reverte
P5	Contabilità coerente	<code>totalReceived == registrationCount * REGISTRATION_FEE</code>
P6	Revert atomico	Una chiamata invalida non lascia <code>registered</code> , <code>registrationCount</code> o <code>totalReceived</code> modificati a metà

P1 IN PRATICA: TESTARE SOLO «TROPPO POCO» NON BASTA

0 ether → fallisce

0.5 ether → fallisce

1 ether → può riuscire

2 ether → fallisce

6 · SCEGLIERE L'INVARIANT GIUSTO

Un invariant deriva da ciò che il contratto controlla

LA USEREMO

`totalReceived = registrationCount * fee`

Deriva dal modello applicativo: ogni registrazione riuscita costa esattamente una fee costante.

NON COME INVARIANT DI BUSINESS

`address(registry).balance = totalReceived`

Più fragile: nell'EVM l'Ether può arrivare a un address senza passare dalla funzione applicativa attesa.

Non da un'uguaglianza intuitiva che pretende più di quanto la EVM garantisca.

Installa o aggiorna, poi crea lesson-02

```
# installa o aggiorna
curl -L https://getfoundry.sh/install | bash
foundryup

# verifica
forge --version
anvil --version
cast --version

# crea il progetto
forge init lesson-02
cd lesson-02
```

```
# foundry.toml
[profile.default]
src = "src"
test = "test"
out = "out"
libs = ["lib"]
solc_version = "0.8.37"
optimizer = true
optimizer_runs = 200
```

Se Foundry è già installato, foundryup lo aggiorna. Sostituisci contratto e test del template con quelli della lezione; forge config mostra la configurazione risolta.

7.4 • TEST/FIXEDFEEREGISTRY.T.SOL

La suite: harness, account, stato di partenza

```
import {Test} from "forge-std/Test.sol";
import {FixedFeeRegistry} from "../src/FixedFeeRegistry.sol";

contract FixedFeeRegistryTest is Test {
    FixedFeeRegistry internal registry;

    address internal constant ALICE = address(0xA11CE);
    address internal constant BOB = address(0xB0B);
    uint256 internal constant FEE = 1 ether;

    function setUp() public {
        registry = new FixedFeeRegistry();
        vm.deal(ALICE, 10 ether);
        vm.deal(BOB, 10 ether);
    }
}
```

SETUP()

Un registry nuovo e 10 ether fittizi ad Alice e Bob, prima di ogni test.

TEST_INITIALSTATE

Il primo test fotografa la partenza: nessuno registrato, contatori a zero, fee di 1 ether.

7.4 · I TEST POSITIVI

Test positivi, più una proprietà condivisa

```
function test_Register_SucceedsWithExactFee() public {
    vm.prank(ALICE);
    registry.register{value: FEE}();

    assertTrue(registry.registered(ALICE));
    assertEquals(registry.registrationCount(), 1);
    assertEquals(registry.totalReceived(), FEE);
    _assertAccountingProperty();
}

function _assertAccountingProperty() internal view {
    assertEquals(
        registry.totalReceived(),
        registry.registrationCount() * FEE
    );
}
```

Due account, due registrazioni

Alice e Bob si registrano entrambi: tutti e due risultano registrati, conteggio 2, totale $2 \times \text{FEE}$.

test_Register_TwoDifferentAccountsRemainIndependent

La helper contabile

_assertAccountingProperty() verifica P5 alla fine di ogni test, positivo o negativo.

7.4 · I TEST NEGATIVI

Un test negativo: errore esatto, poi lo stato

```
function test_RevertWhen_FeeIsTooHigh() public {
    uint256 sent = 2 ether;
    vm.expectRevert(abi.encodeWithSelector(
        FixedFeeRegistry.ExactFeeRequired.selector, sent, FEE
    ));
    vm.prank(ALICE);
    registry.register{value: sent}();

    assertFalse(registry.registered(ALICE));
    assertEquals(registry.registrationCount(), 0);
    assertEquals(registry.totalReceived(), 0);
    _assertAccountingProperty();
}
```

1 · ERRORE ESATTO

Selector e argomenti: sent e FEE.

2 · POI LO STATO

Nessuna registrazione, contatori a zero, P5 ancora vera.

3 · STESSO SCHEMA

FeeIsZero con 0, FeeIsTooLow con 0.5 ether, AccountRegistersTwice con AlreadyRegistered.

Il ciclo di lavoro, comando per comando

<code>forge fmt</code>	formatta: meno rumore stilistico nelle review
<code>forge build</code>	deve finire senza errori prima di leggere qualsiasi test
<code>forge test</code>	esegue la suite: tutti PASS, zero failure
<code>forge test --match-test test_RevertWhen_FeeIsTooHigh</code>	un solo test: ciclo rapido e mirato
<code>forge test --match-contract FixedFeeRegistryTest</code>	un solo contratto di test
<code>forge test -vvv</code>	trace dei test falliti
<code>forge test -vvvv --match-test <nome></code>	trace anche dei test riusciti
<code>forge test --debug --match-test <nome></code>	debugger: fino all'esecuzione EVM
<code>forge coverage</code>	quali rami non hai mai esercitato

Non memorizzare i numeri di gas: dipendono da compilatore, optimizer e toolchain. Conta il comportamento.

7.8 · LEGGERE LE TRACE

Una trace è una call tree, non un log di debug

SUCCESSO · -VVVV

```
FixedFeeRegistryTest
| vm.prank(ALICE)
|
+→ register{value: 1 ether}()
    +→ legge registered[ALICE]
    +→ scritture storage
    +→ emit Registered
    +→ return
```

REVERT · -VVVV

```
FixedFeeRegistryTest
| vm.expectRevert(...)
| vm.prank(ALICE)
|
+→ register{value: 2 ether}()
    +→ fee ≠ expected
    +→ REVERT ExactFeeRequired(...)
```

Cerca chiamante, valore, confine della call e punto del revert. Servirà quando arriveranno chiamate annidate e reentrancy.

8 • TEST NEGATIVI

Cinque casi, volutamente ridondanti con l'happy path

Caso	Input	Atteso	Perché conta
1 • Zero Ether	0 ETH	ExactFeeRequired; nessuna registrazione	il confine inferiore più ovvio
2 • Fee insufficiente	0.5 ETH	ExactFeeRequired	distingue == da controlli che accettano importi troppo bassi
3 • Fee eccessiva	2 ETH	ExactFeeRequired	se il requisito è «esattamente», va testato anche il sopra
4 • Doppia registrazione	seconda call con 1 ETH	AlreadyRegistered; conteggio 1, totale 1 ETH, Alice registrata	il fallimento non corrompe la contabilità
5 • Separazione utenti	Alice registrata	registered[BOB] falso, poi Bob si registra	lo stato resta per account, non globale

Rendiamo sbagliato il contratto, apposta

CORRETTO

```
if (msg.value  $\neq$  REGISTRATION_FEE) {  
    revert ExactFeeRequired(...);  
}
```

Tutto ciò che non è esattamente 1 ETH viene rifiutato.

MUTANTE

```
if (msg.value < REGISTRATION_FEE) {  
    revert ExactFeeRequired(...);  
}
```

Meno di 1 ETH: rifiuta. 1 ETH o più: accetta.

La specifica diceva «esattamente 1 ETH»: è un bug di implementazione rispetto al requisito.

9.2 · 9.3 · QUALE TEST SE NE ACCORGE?

Solo il test sopra la soglia distingue il mutante

```
forge test --match-test test_RevertWhen_FeeIsTooHigh -vvvv
```

Test	Con il mutante
test_Register_SucceedsWithExactFee	passa ancora: 1 ETH resta accettato
test_RevertWhen_FeeIsTooLow	passa ancora: 0.5 ETH resta rifiutato
test_RevertWhen_FeeIsTooHigh	fallisce: 2 ETH non reverte, l'expectRevert non è soddisfatto

I CONFINI SI TESTANO DA ENTRAMBI I LATI

Un test è utile quando distingue almeno un'implementazione plausibilmente sbagliata da quella corretta.

Dal bug al test di regressione, in cinque passi

1 · REQUISITO

`fee = 1 ETH`

2 · MUTANTE

`fee ≥ 1 ETH`

3 · TEST CHE LO SCOPRE

2 ETH deve
revertire

4 · CORREZIONE

`fee ≠ 1 ETH`
→ revert

5 · REGRESSIONE

il test resta
nella suite

Ripristinato il controllo `!=`, la suite torna verde. `test_RevertWhen_FeeIsTooHigh` diventa un test di regressione: impedisce che «fee esatta» torni, per errore, a essere «fee minima».

È la sequenza che useremo nelle prossime lezioni, sulle vulnerabilità vere e proprie.

Contratto e suite: specifica, chiamante, revert

Specifica

- Riesco a dire in una frase cosa deve garantire ogni funzione importante?
- «Esattamente», «almeno», «al massimo», «una sola volta», «solo owner» diventano test distinti?

Caller e valore

- Esistono test con chiamanti differenti?
- I test verificano davvero msg.sender, o tutte le chiamate partono dal test contract?
- Le funzioni payable sono testate con zero, valore valido e valori fuori intervallo?

Revert

- I casi vietati sono testati?
- Il test verifica l'errore corretto, quando è utile?
- Dopo il revert si verifica che lo stato non sia cambiato?

Lo stato, e la qualità della suite

Stato

- Contatori e mapping sono controllati dopo ogni transizione importante?
- La suite verifica la separazione tra utenti?
- Esistono proprietà contabili esprimibili come uguaglianze?

Qualità della suite

- Posso immaginare una piccola mutazione sbagliata che tutti i test lascerebbero passare?
- Uso la coverage come strumento diagnostico o come falsa prova di sicurezza?
- Ogni bug corretto ha un test di regressione?

11 • ESERCIZI

Prevedi, poi verifica con Foundry

1 • Stato invariato dopo revert

In `FeeIsTooHigh` salva i saldi di ALICE e del registry prima della call: cosa osservi dopo il revert?

2 • Test che manca

`Fee` sbagliata, revert; poi `fee` corretta, successo. Un tentativo fallito non consuma il diritto.

3 • Mutazione sulla registrazione

Commenta `registered[msg.sender] = true`: prevedi quali test falliscono, poi verifica.

4 • Mutazione sul contatore

`registrationCount += 2`: quali test lo rilevano? La helper contabile aggiunge valore?

5 • Errore troppo generico

`expectRevert()` senza dati e un revert precedente non correlato: il test passa lo stesso.

6 • Specifica prima del codice

Stop dopo 100 registrazioni: requisiti, proprietà, nomi dei test, casi di confine. Niente codice.

7 • Audit della suite

Con due soli test, trova almeno tre implementazioni errate che sopravvivono.

Nessuna soluzione qui

Prima formula la proprietà, poi scrivi il test.

Otto idee da portare via

1 · I test nascono dalla specifica

«Il codice garantisce ciò che il protocollo richiede?»

2 · L'happy path è solo l'inizio

Rifiutare bene input e transizioni vietate è sicurezza.

3 · `vm.prank` e `vm.deal` costruiscono il contesto

Identità e fondi fittizi, nessuna rete reale.

4 · Un revert si verifica anche negli effetti

Lo stato importante deve restare coerente.

5 · Proprietà forti, non controlli superficiali

`totalReceived == registrationCount * fee`

6 · Coverage non significa sicurezza

Dice cosa hai eseguito, non se le proprietà sono giuste.

7 · Ogni bug corretto lascia un test di regressione

Senza, il bug può tornare.

8 · Quale mutazione sbagliata sopravvive?

Se tutti i test passano, la suite ha una lacuna.

Da dove viene ogni affermazione

Fonte	Per che cosa
Foundry · sito e installazione	installazione; ruolo di Forge, Anvil, Cast, Chisel
Writing Tests · forge test	setUp, naming, prefisso test, filtri, verbosity
Forge Standard Library	forge-std/Test.sol e istanza vm
prank · expectRevert · interfaccia deal	semantica e firme dei cheatcode
Debugger · Traces · Configuration	--debug, call tree, foundry.toml
ethereum.org · Testing · Security	test, coverage; testing necessario ma non sufficiente
Solidity · Error handling · Custom Errors	semantica dei revert, custom errors nei test
Solidity 0.8.37 Release Announcement	baseline del compilatore e bugfix